

Домашнее задание 1

Прочитай о реальных примерах, где недостаточное тестирование привело к серьёзным последствиям, как в случае с Therac-25 или Boeing 737 MAX. Напиши эссе о том, какие принципы тестирования могли бы предотвратить эти проблемы.

Кейс 1: Therac-25 (1985–1987)

История: Therac-25 — это медицинский линейный ускоритель для лучевой терапии рака, разработанный канадской компанией AECL . В отличие от предыдущих моделей (Therac-6 и Therac-20), этот аппарат полностью полагался на компьютерный контроль. Разработчики убрали аппаратные блокировки (hardware interlocks), которые раньше служили последней линией обороны, переложив всю ответственность за безопасность на программное обеспечение .

Последствия: Из-за гонки состояний (race condition) и ошибки переполнения переменной в софте, при определенной последовательности действий оператора машина выдавала не рассеянный луч для терапии, а смертельный импульс электронов в полную мощность. Пациенты описывали это как «удар током», после чего следовали тяжелые радиационные ожоги и смерть. В период с 1985 по 1987 год произошло как минимум 6 инцидентов, приведших к смертельным исходам .

Кейс 2: Boeing 737 MAX (2018–2019)

История: Чтобы конкурировать с Airbus и экономить топливо, Boeing установил на старый пласт 737 новые, более крупные и тяжелые двигатели. Из-за габаритов двигатели сместили вперед и вверх, что изменило аэродинамику самолета, вызвав риск сваливания на взлете.

Вместо перепроектирования планера или дорогостоящего переобучения пилотов, Boeing написал программу MCAS (Maneuvering Characteristics Augmentation System), которая автоматически опускала нос самолета

Последствия: MCAS полагалась всего на один датчик угла атаки (AoA). В полетах Lion Air и Ethiopian Airlines этот датчик дал сбой. Система, думая, что самолет падает, бесконтрольно уводила лайнер в пике. Пилоты не знали о существовании MCAS (этого не было в документации), поэтому не могли отключить систему. Две катастрофы унесли жизни 346 человек .

Основные нарушенные принципы тестирования и разработки

1. Никогда не убирайте аппаратную защиту, если заменяете ее софтом (Принцип разнородности / Fail-safe):

В Therac-25 убрали физические блокировки. Если софт — сложная штука, в которой есть баги, «железо» должно уметь спасти жизнь, даже если мозг (ПО) «сошел с ума». *Чего не хватало: Hardware interlocks.* .

2. Сквозное тестирование, а не только «счастливый путь» (Экстремальное тестирование / Stress testing):

В Therac-25 баг проявлялся, только если оператор очень быстро набирал команды (гонка состояний). Тестировщики, вероятно, нажимали клавиши медленно. В Boeing тестировали MCAS на симуляторах с пилотами-испытателями, но не учли, что реальные пилоты могут не знать о системе и паниковать . *Чего не хватало: Тестирование гонок состояний (Race conditions) и usability-тестирование с реальными пользователями.*

3. Принцип «Надейся на лучшее, готовься к худшему» — физическая отказоустойчивость (Redundancy):

Boeing использовал сигнал всего с одного датчика угла атаки. В авиации это нонсенс. Если бы MCAS сравнивала показания двух (или трех)

датчиков, она бы поняла, что датчик сломан, и отключилась. *Чего не хватало: Проверки согласованности данных (Cross-validation).* .

4. Повторное использование кода — это не баг-фикс (Regression testing):

Программисты Therac-25 использовали старый код от Therac-6/20, который раньше был безопасен. Но они не учли, что раньше этот код «подстраховывался» железом, а теперь работает один. Код нужно тестировать в новой среде, даже если он старый. *Чего не хватало: Регрессионного тестирования в новой архитектуре.* .

5. Прозрачность и документация ошибок (Error Handling & Transparency):

Therac-25 выдавал оператору просто слово «MALFUNCTION» (Неисправность) и цифру. Расшифровки в документации не было. В Boeing скрыли само существование MCAS от пилотов. Если пользователь не понимает сообщение об ошибке, он не может исправить ситуацию. *Чего не хватало: Человеко-читаемые сообщения и полнота обучающей документации.*

Существует несколько классификаций принципов тестирования. Самый авторитетный источник — это ISTQB (International Software Testing Qualifications Board). Я перечислю семь классических принципов, а затем добавлю важные «промышленные» принципы, которые вытекают из трагедий (Therac-25, Boeing), но не всегда входят в базовый учебник.

Семь фундаментальных принципов тестирования (по ISTQB)

1. Тестирование показывает наличие дефектов, но не доказывает их отсутствие (Testing shows presence of defects)

Успешный тест — это тот, который нашел баг. Даже если все тесты

прошли, это не значит, что багов нет — значит, вы их просто не нашли.

2. Истощающее тестирование невозможно (Exhaustive testing is impossible)

Нельзя проверить все комбинации вводов и состояний (кроме тривиальных программ). Вместо этого используют риски и приоритеты.

3. Раннее тестирование (Early testing)

Находить баг на этапе требований (дешево), а не после релиза в проде (дорого и смертельно для Boeing). Чем раньше вы включили тестирование, тем лучше.

4. Кластеризация дефектов (Defect clustering)

80% проблем обычно сосредоточены в 20% модулей системы. Принцип Парето работает и здесь.

5. Парадокс пестицида (Pesticide paradox)

Если повторять одни и те же тесты снова и снова, они перестают находить новые баги. Тесты нужно менять, дополнять, переписывать.

6. Тестирование зависит от контекста (Testing is context dependent)

Что работает для тестирования интернет-магазина, не работает для управления ядерным реактором или автопилотом. Контекст решает всё.

7. Заблуждение об отсутствии ошибок (Absence-of-errors fallacy)

Можно найти и исправить все баги, но создать систему, которая никому не нужна или не решает проблем пользователя. Ошибки $\neq$ полезность.

Принципы, которые «добавила» реальная жизнь (Therac-25, Boeing, медицина, авиация)

Эти принципы не всегда есть в ISTQB, но они критически важны для систем, где цена ошибки — жизнь.

Принцип		Суть
Принцип разнородности (Diversity)		Нельзя полагаться на один тип защиты. Если софт отвечает за безопасность, параллельно должна быть аппаратная блокировка. Разные технологии должны дублировать друг друга.
Принцип единственного источника отказа (Single Point of Failure)		Никогда не доверяйте критическое решение одному датчику, одной переменной, одному каналу. Если данных два — их нужно перекрестно проверять.
Принцип (Безопасный отказ)	fail-safe	Когда система ломается, она должна переходить в состояние, безопасное для человека (выключиться, отпустить тормоза, поднять нос самолета). Сбой ≠ катастрофа.

Принцип человеческого фактора (Usability in failure)	Если система показывает ошибку «MALFUNCTION 0», а пользователь не знает, что делать — система сломалась дважды. Сообщения об ошибках должны быть понятны и вести к действию.
Принцип регрессионной защиты (Regression ≠ Trust)	Если вы взяли старый, проверенный код, но поместили его в новое окружение (убрали железную блокировку), старый код становится новым и опасным. Перетестируйте всё.
Принцип независимости (Independence)	Разработчик не может объективно тестировать свой код. Нужен отдельный тестировщик, а для безопасности — внешний аудит.

Короткая памятка для эссе или экзамена

Если вас попросят назвать главные принципы тестирования, перечислите эти 5 ключевых:

1. Раннее тестирование — ищи проблемы на этапе требований и дизайна.

2. Невозможность полной проверки — используйте риски и приоритеты, не пытайтесь проверить всё.
3. Кластеризация дефектов — 20% модулей дают 80% проблем.
4. Парадокс пестицида — одни и те же тесты перестают работать, их надо менять.
5. Безопасный отказ (fail-safe) — если система падает, она должна падать безопасно.

А для критически важных систем (как Therac-25 и Boeing) добавьте ещё три:

- Аппаратное дублирование софта.
- Перекрестная проверка датчиков (нет единому источнику истины).
- Человеко-читаемые ошибки и обучение пользователя.

Эссе. Цена самоуверенности: Как Therac-25 и Boeing 737 MAX переписали законы тестирования

В истории инженерии есть уроки, которые мы учим снова и снова. Самые страшные катастрофы происходят не тогда, когда программисты глупы, а когда инженеры самоуверенны. Две трагедии разделяет тридцать лет и разные отрасли — медицинская физика и авиастроение, — но их корни удивительно похожи. Аппарат лучевой терапии Therac-25 и авиалайнер Boeing 737 MAX стали жертвами одной иллюзии: иллюзии, что программный код может быть идеальным, а аппаратные железки — лишними. Последствия оказались фатальными: десятки погибших от радиации и сотни тел в Индонезии и Эфиопии. Эти кейсы навсегда изменили подход к тестированию, выдвинув несколько «железобетонных» принципов, нарушать которые человеческая жизнь не позволяет.

Первый и главный урок — это закон подлости, сформулированный на крови пациентов Therac-25. Убирая аппаратные блокировки, разработчики совершили роковую ошибку. Раньше, в модели Therac-20, если софт сходил с ума, железный тумблер физически прерывал цепь питания излучателя. Создатели Therac-25 решили, что софт надежен настолько, что «железо» уже не нужно. Но программирование — это не абсолютная истина, это работа с вероятностью. MCAS в «Боинге» — та же история, только на другом уровне. Компания доверила управление самолетом одному датчику угла атаки, не перепроверяя его данные через второй датчик. Принцип разнородности (не полагайся на единую точку отказа) и физической обратной связи должен быть священным: программный модуль не должен быть судьей, палачом и единственным стражем. Тестирование обязано проверять не только корректность кода, но и наличие «подпорок» — систем, которые остановят катастрофу, когда код даст сбой.

Второй урок касается «слепых зон» тестирования. Ошибка в Therac-25 возникала только при быстрой печати команд на клавиатуре — явление, известное как race condition (гонка состояний). Стандартные тесты не выявили проблему, потому что тестировщики не нажимали клавиши так быстро, как уставший оператор. В Boeing 737 MAX тестирование признавало систему безопасной, если пилот реагировал на ложное пикирование за 3–4 секунды. Но реальные пилоты, не знавшие о существовании системы MCAS, тратили на осознание ситуации более 10 секунд — и разбивались. Эти примеры учат нас, что тестирование должно учитывать человеческий фактор и нетипичные сценарии (Edge Cases). Нельзя проверять систему в стерильной лаборатории, нужно симулировать хаос, усталость, незнание и панику.

Третий, самый циничный принцип, связан с экономикой и гордыней. Обе компании пошли по пути наименьшего сопротивления. AECL сэкономила на независимом аудите кода. Boeing, чтобы не переучивать пилотов (что стоило бы миллиарды), просто скрыла факт существования MCAS от экипажей. Тестирование здесь превратилось в формальность для галочки, а не в инструмент поиска правды. Когда в Therac-25 обнаружили первую проблему, компания заявила об улучшении безопасности «на пять порядков» без доказательств. После первого крушения Boeing отрицал вину, списывая все на ошибку пилотов. Истинное тестирование не может существовать внутри корпоративной культуры, где безопасность стоит дешевле, чем репутация.

Эти два кошмара подарили миру новую аксиому: «Тестирование — это не поиск багов, это поиск слепых зон системы». Чтобы предотвратить повторение Therac-25, сегодня в индустрии внедряют стандарты вроде IEC 62304 (для медицинского ПО), требующие тотального трейсинга требований и моделирования отказов. Чтобы исправить ошибки Boeing, регуляторы больше никогда не позволят производителю сертифицировать систему, где критическая функция завязана на один датчик без аппаратного дублирования.

Будем честны: и Therac-25, и Boeing 737 MAX были технически совершенны на 99,9%. Но беда в том, что авиапассажир или онкобольной — это не кофемолка. Сбой в 0,1% стоит человеческой жизни. Тестирование жизненно-критичных систем должно быть направлено не на подтверждение работы кода, а на поиск доказательств того, что система *не убьет* человека, когда этот код даст сбой. Ведь как показала история, код ошибается всегда. Вопрос лишь в том, есть ли у нас смелость и ресурсы проверить, что случится, когда это произойдет.

**Лист проверки базовой функциональности: Форма регистрации
пользователя в instagram**

Дата: _____ Тестировщик: _____ Версия сборки: _____

Обозначения: + Пройдено | - Не пройдено | ! Внимание (есть нюанс)

№	Категория	Пункт проверки	Результат	Примечание
1	Позитив	Успешный ввод пароля, логина и никнейма	+	
	Позитив	При случайном вводе на другом языке система считывает и допускает	-	Баг: не чувствителен к регистру
	Позитив	При включенном Caps Lock система считывает и допускает	-	Баг: не чувствителен к регистру
	Негатив	Пустые поля (логин пустой, пароль есть и наоборот) - кнопка не активна	+	
	Негатив	При регистрации никнейма уже имеется	!	Доступность для пользователя : короткое

		такое имя - ошибка		объяснение в плашке ошибки, почему не принимают никнейм
	Негатив	Пустое поле никнейма - ошибка	!	Доступность для пользователя : короткое объяснение в плашке ошибки, почему не принимают никнейм
	Негатив	Пробелы в полях	-	Считаются ли пробелы заполненны м полем?
	Границы	Длина пароля(от 6 до 30 символов) - принимается	+	
	Границы	Специальны е символы (!№@._) - принимается	-	Символы и эмоджи, не включенные в список, не должны проходить. Пробелы в пароле разрешены?
	Границы	Дата рождения больше 2025 года не принимается	+	

	Интерфейс	Плашка “Ошибка” при неверном пароле/ логине/ пустом поле/ пробелах в полях	!	Не уходит после повторного исправления
	Интерфейс	Доступные короткие объяснения ошибок пользовател ю	!	Не все ошибки объясняются
	Интерфейс	Пароль зашифрован звездочками при вводе и есть возможность его посмотреть нажав на “Глаз”	+	
	Интерфейс	Подтвержде ние почты/ телефона	+	
	Интерфейс	Согласие с правилами сообщества	-	Баг: не работает кнопка “Согласитьс я”

Эталонный чек-лист (для сравнения)

Ниже — как выглядел бы идеальный чек-лист только для регистрации Instagram (без смещения со входом):

№	Категория	Пункт проверки
1	Позитив	Регистрация с валидными: email/телефон, полное имя, никнейм, пароль (6-30 символов), дата рождения (>13 лет), согласие с правилами → аккаунт создан
2	Позитив	После регистрации приходит код подтверждения на email/телефон, код принимается
3	Негатив	Никнейм уже существует → ошибка «Это имя занято» с предложением альтернатив
4	Негатив	Пароль короче 6 символов → ошибка
5	Негатив	Пароль длиннее 30 символов → ошибка

6	Негатив	Пароль только из цифр (слишком простой) → ошибка «ненадежный пароль»
7	Негатив	Поля содержат только пробелы → ошибка
8	Негатив	Дата рождения указывает возраст <13 лет → ошибка «Вы слишком молоды»
9	Границы	Спецсимволы в пароле (!@#\$%^&*) — принимаются
10	Границы	Эмодзи в никнейме — НЕ принимаются
11	Границы	Никнейм длиной 1 символ — минимально допустимый (буква)
12	Границы	Никнейм длиной 30 символов — максимально допустимый
13	Интерфейс	Все ошибки подсвечивают конкретное поле и содержат понятный текст

14	Интерфейс	Ошибка исчезает, когда пользователь начинает исправлять поле	
15	Интерфейс	Кнопка «Зарегистрироваться» неактивна, пока не заполнены все обязательные поля	
16	Интерфейс	Пароль звездочками, есть «Показать»	скрыт кнопка